GROUP ID: 62

A PROJECT REPORT ON

CHHAAP: LAYOUT-FIRST DIFFUSION WITH SYMBOL SURVIVAL CHECKS FOR WARLI-INSPIRED PRODUCT DESIGN

SUBMITTED TO THE PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
AN AUTONOMOUS INSTITUTE, PUNE

IN THE FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE
OF
B. TECH. IN "COMPUTER SCIENCE AND ENGINEERING (AI & ML)"
FOR THE SUBJECT
GENERATIVE AI MDM CAPSTONE PROJECT
UNDER THE DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

SUBMITTED BY

KESHAV SHETSHRINGI PRN: 123B1B166
AADI KANCHANKAR PRN: 123B1B162
SWAPNIL KHILLARE PRN: 123B1B168
PRATHAM KOLETTIWAR PRN: 123B1B169

UNDER THE GUIDANCE OF
PROF. PALLAVI NIKUMBH (DHADE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)
PCET'S PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
Sector No. 26, Pradhikaran, Nigdi, Pimpri-Chinchwad, PUNE – 411044

2026-2027

PCET'S PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

Sector No. 26, Pradhikaran, Nigdi, Pimpri-Chinchwad, PUNE 411044

2026-2027

CERTIFICATE

This is to certify that the project report entitled

"CHHAAP: LAYOUT-FIRST DIFFUSION WITH SYMBOL SURVIVAL CHECKS FOR WARLI-INSPIRED PRODUCT DESIGN"

Submitted by

KESHAV SHETSHRINGI PRN: 123B1B166

AADI KANCHANKAR PRN: 123B1B162

SWAPNIL KHILLARE PRN: 123B1B168

PRATHAM KOLETTIWAR PRN: 123B1B169

are bonafide students of this institute and the work has been carried out by them under the supervision of Prof. Pallavi Nikumbh (Dhade) and it is approved for the partial fulfillment of the requirement of Pimpri Chinchwad College of Engineering an autonomous institute, for the award of the B. Tech. degree in "Computer Science and Engineering (AI & ML)".

(Prof. Pallavi Nikumbh (Dhade))
Guide
Department of CSE (AI & ML)

(Prof. Dr. Anuradha Thakare)
Head,
Department of CSE (AI & ML)

Place: Pune

Date:

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Prof. Pallavi Nikumbh (Dhade), for her guidance, encouragement and critical feedback at every stage of this project. Her suggestions on evaluation and on the ethical framing of the work shaped this report.

We thank Prof. Dr. Anuradha Thakare, Head of the Department of Computer Science and Engineering (AI & ML), for providing the facilities and the opportunity to carry out this capstone project, and the faculty and staff of the department for their support.

We acknowledge the Warli community of the north Konkan coast, whose art inspired this work, and the authors of the open models, datasets and documentation on which this project is built. Finally, we thank our families and friends for their constant support.

Keshav Shetshringi
Aadi Kanchankar
Swapnil Khillare
Pratham Kolettiwar

ABSTRACT

Printed Warli products need structured layouts, designed colour and, for repeating products, tiles without a visible join, and buyers rarely learn what the motifs on them mean. General text-to-image models produce framed pictures with cut-off figures, tile with visible seams, and give no reliable account of which symbols they drew. This project presents Chhaap, a local generative design studio that turns a short written brief into a finished, print-ready Warli-inspired product design together with a meaning card that explains every symbol on it.

A planner maps the brief to a validated plan. A layout engine places symbols from a sourced lexicon of 25 motifs at product size in one of four templates (tarpa spiral, registers, medallion and all-over buti), colour comes from the layout's layers, and Stable Diffusion 1.5 with a scribble ControlNet, an LCM-LoRA and a Warli LoRA trained by us paints only the line work. Circular padding in every convolution makes repeating tiles wrap. A survival check compares each placed symbol with the generated output and repairs losses, first by rerunning with a stronger ControlNet and then by restoring the vector symbol.

Over 16 hand-painted pages (852 symbols) diffusion kept 0.587 of hero and support symbols on its first pass; repair raised this to 1.000 at a cost of 141.2 seconds per page on a laptop CPU, against 0.152 seconds for vector-only rendering. Symbols with more drawing primitives were lost more often (Spearman rho = -0.474, p = 0.0349), and rotated ring dancers survived far less often than upright symbols (0.468 against 0.828). Structured templates spread ink more evenly than random scatter, and circular padding turned framed panels into continuous textures. The rules-based planner satisfied 0.944 of the constraints stated in 20 test briefs. Every export is labelled as AI-generated and Warli-inspired, and an artist-licensed adapter with royalties is proposed for deployment.

Keywords: Generative AI, latent diffusion, ControlNet, LoRA, tileable textures, cultural heritage, Warli.

TABLE OF CONTENTS

Sr. No.	Title of Chapter	Page No.
	List of Abbreviations	vii
	Nomenclature	viii
	List of Figures	ix
	List of Tables	x
01	**Introduction**	**1**
	1.1 Overview of Generative AI	1
	1.2 Project Background	1
	1.3 Motivation	2
	1.4 Problem Statement	2
	1.5 Objectives	2
	1.6 Scope of the Work	3
02	**Literature Survey**	**4**
	2.1 Review of Recent Literature	4
	2.2 Generative AI: Existing Approaches	4
	2.3 Review of Related Generative AI Systems	5
	2.4 Existing Solutions / State-Of-The-Art Methods	5
	2.5 Gap Identification	5
	2.6 Common Findings From Literature	6
03	**Requirements Specification**	**7**
	3.1 Hardware Requirements	7
	3.2 Software Requirements	7
	3.3 Functional Requirements	7
	3.4 Non-Functional Requirements	8
	3.5 Generative AI Model Requirements	8
	3.6 Dataset / Knowledge-Base Requirements	8
04	**System Design**	**10**
	4.1 Proposed System Architecture	10
	4.2 System Block Diagram	10
	4.3 System Workflow	11
	4.4 Mathematical Model	11
05	**Project Implementation**	**13**
	5.1 Overview of Project Modules	13
	5.2 Data Collection and Preprocessing	13
	5.3 Generative AI Model Selection	13
	5.4 Model Configuration / Fine-Tuning	14
	5.5 Project Modules	15
	5.6 Tools and Technologies Used	16
	5.7 Algorithm Details	16
06	**Software Testing**	**18**
	6.1 Test Cases	18
	6.2 Generative AI Output Testing	18
	6.3 Performance Evaluation	18
07	**Results and Discussion**	**20**
	7.1 Project Outcomes	20

	7.2 Result Analysis	20
	7.3 Generated Output Analysis	21
	7.4 Performance Metrics	23
	7.5 Accuracy / Relevance / Quality Evaluation	24
	7.6 Validation of Results	24
	7.7 Comparison with Existing Approaches	25
	7.8 Screenshots of the Implemented System	26
08	**Conclusion and Future Work**	**28**
	8.1 Conclusion	28
	8.2 Limitations of the Proposed System	28
	8.3 Future Work	28
	8.4 Applications	29
	References	**30**

LIST OF ABBREVIATIONS

ABBREVIATION	ILLUSTRATION
AI	Artificial Intelligence
CLIP	Contrastive Language-Image Pre-training
BLIP	Bootstrapping Language-Image Pre-training
CPU	Central Processing Unit
CV	Coefficient of Variation
DPI	Dots Per Inch
FID	Frechet Inception Distance
GAN	Generative Adversarial Network
GI	Geographical Indication
GPU	Graphics Processing Unit
IoU	Intersection over Union
KID	Kernel Inception Distance
LCM	Latent Consistency Model
LoRA	Low-Rank Adaptation
RSS	Resident Set Size
SD	Stable Diffusion
SDR	Seam Discontinuity Ratio
SVG	Scalable Vector Graphics
TAESD	Tiny AutoEncoder for Stable Diffusion
UNet	U-shaped convolutional Network
VAE	Variational AutoEncoder

NOMENCLATURE

SYMBOL	MEANING
T	Generated tile of height H and width W
G	Sobel gradient magnitude of the 2 x 2 mosaic
SDR	Mean seam-band gradient divided by mean interior-band gradient
M_c, M_o	Composition ink mask and output ink mask inside an element box
tau	Survival threshold on IoU (0.5)
s_c	ControlNet conditioning scale (0.7, +0.15 per rerun)
alpha	LoRA strength (0.8)
rho	Spearman rank correlation coefficient

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE NO.
4.1	Proposed system architecture of Chhaap	10
4.2	Block view of one generation: control image, step previews, decoded tile and two-tone reduction	10
5.1	LoRA checkpoints (rows: 500, 1000, 1500 steps) against strength (columns: 0.4, 0.8, 1.2)	14
7.1	Vector-only designs from four briefs: saree set, diary cover, phone wallpapers and a frame with a Mandana border	20
7.2	Share of each symbol surviving the first diffusion pass over 16 pages	21
7.3	Four of the 16 runs: composition, stylised output and repaired ink	22
7.4	2 x 2 mosaics per condition (rows a to e); seam lines run through the centre of each mosaic	23
7.5	Chhaap web interface: brief, plan card, press settings and design output	26
7.6	The A3 anniversary brief, hand-painted with repair, on a wall-frame mockup	27

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE NO.
2.1	Comparison of existing approaches	5
3.1	Hardware requirements	7
3.2	Generative models used	8
5.1	Tools and technologies	16
6.1	Test cases	18
6.2	Cost on the test laptop (CPU only), 16 images per condition	18
7.1	Symbol survival and time per page (16 pages)	23
7.2	Evenness of ink (CV over 4 x 4 grid, lower is more even), 16 seeds	24
7.3	Tiling, post-hoc |log SDR| (0 is seamless), 16 tiles	24
7.4	Comparison with a general text-to-image baseline	25

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW OF GENERATIVE AI

Generative Artificial Intelligence refers to models that learn the distribution of a data set and can draw new samples from it: text, images, audio or code. Earlier generative models such as variational autoencoders and generative adversarial networks (GANs) showed that neural networks can synthesise realistic images, but they were hard to train and to control. Diffusion models changed this. They learn to reverse a gradual noising process, and at generation time they start from random noise and remove it step by step, guided by a text prompt.

Latent diffusion models run this process in the compressed latent space of an autoencoder, which made 512-pixel text-to-image generation affordable on ordinary hardware. Stable Diffusion 1.5 is the open model of this family. Around it a set of adaptation methods has grown: Low-Rank Adaptation (LoRA) teaches the model a new style from a few hundred images, ControlNet conditions it on a spatial image such as a scribble or an edge map, and consistency distillation (LCM-LoRA) cuts sampling from 25 or more steps to 4 to 8. Together these allow a style-specific, spatially controlled generator to run on a laptop.

1.2 PROJECT BACKGROUND

Warli painting comes from the Warli community of the north Konkan coast of Maharashtra. Its figures are drawn in white rice paste on a red ochre wall: people as two triangles joined at the tip, the sun and moon as circles, and a sacred square called the chauk at the centre of ritual paintings [1], [2]. The style now reaches buyers mostly as printed goods: textiles, phone cases, diaries and wallpaper.

Printing an all-over pattern by machine needs a tile whose right edge continues into its left edge and whose top continues into its bottom. A designer working by hand can draw such a tile. A general text-to-image model, asked for "a Warli pattern", usually cannot: the output has a frame of cut-off figures, and repeating it shows a grid

of seams. Published Warli prints are also organised in specific ways (bands, dance rings, medallions, borders and fillers), with colour designed in three to five tones.

1.3 MOTIVATION

Warli symbols carry meanings that the community and its documenters have recorded, and a buyer who sees a printed tarpa dance has no way to learn what it is. If a model generates the picture first and a captioner guesses its symbols afterwards, any explanation printed with the product can be wrong. We wanted a tool in which the explanation is correct by construction, the designs are structured like real Warli products, and the generative model is used only where it adds value.

A second motivation is ethical. Warli painting holds a Geographical Indication, and style adapters trained on scraped images take value from a community without its consent. A transparent, local tool that labels its outputs and proposes an artist-licensed model is a better starting point than an opaque online generator.

1.4 PROBLEM STATEMENT

To design and implement a local generative AI system that converts a short natural-language brief into a print-ready, Warli-inspired product design (wall frame, saree set, diary cover, wallpaper, phone wallpaper or tee) with a structured layout, designed colour and seamless repeats where needed, that verifies every important symbol survives diffusion, and that produces a meaning card listing only the symbols actually present, each linked to a published source.

1.5 OBJECTIVES

- To build a sourced lexicon of Warli symbols, each with a drawable geometric recipe and a public source URL.
- To train a Warli style LoRA on Stable Diffusion 1.5 from a cleaned, documented image set.
- To design a layout engine with product-specific templates, layered colour and seamless lattice repeats.

- To use ControlNet-guided diffusion with circular padding to paint only the line work over the layout.
- To check per-symbol survival after diffusion and repair lost symbols automatically.
- To generate meaning cards from lexicon rows of surviving symbols, so that no language model writes meanings.
- To evaluate tiling, style, symbol survival, layout evenness, cost and planner faithfulness quantitatively.

1.6 SCOPE OF THE WORK

The system runs fully on a local machine with open model weights; a CPU is sufficient and a GPU makes it faster. It covers Warli (with a Mandana border pack as a second style), six product types, four layout templates, eight palettes and export at 300 DPI as PNG and SVG with mockups. Training images come from a scraped research data set and are used for research only. The human preference and annotation studies are built into the tool but were not run within the project period. Commercial deployment, artist licensing and verification of meanings by Warli artists are outside the scope of this work.

CHAPTER 2
LITERATURE SURVEY

2.1 REVIEW OF RECENT LITERATURE

Rombach et al. [3] introduced latent diffusion models, which run denoising in the latent space of an autoencoder and made high-resolution text-to-image synthesis affordable; Stable Diffusion 1.5 is the public model of that family on which this project builds. Hu et al. [4] proposed LoRA, which adapts a frozen network by learning a low-rank update to its weight matrices; it is now the usual way to teach a diffusion model a style. Ruiz et al. [5] showed with DreamBooth that binding a subject to a rare token through fine-tuning lets a model reproduce it; we use a trigger word in the same way through LoRA.

Luo et al. [6] distilled a latent consistency model into LCM-LoRA, which lets Stable Diffusion sample in 4 to 8 steps and makes CPU generation usable. TAESD [7] is a tiny distilled autoencoder that decodes latents cheaply enough to preview each sampling step. Zhang et al. [8] introduced ControlNet, which attaches a trainable copy of the diffusion encoder to a frozen model and conditions it on images such as edges, depth or scribbles.

2.2 GENERATIVE AI: EXISTING APPROACHES

Three families of approaches exist for generating images in a given style. Text-only prompting of a general model is cheap but gives little control over composition and almost none over cultural accuracy. Fine-tuning (full fine-tuning, DreamBooth or LoRA) teaches a style but inherits the framing and composition of the training images. Spatially conditioned generation (ControlNet, T2I adapters) fixes where content goes, leaving the model to decide how it looks. Chhaap combines the last two: a LoRA for style and a scribble ControlNet driven by a layout that a program, not the model, decides.

2.3 REVIEW OF RELATED GENERATIVE AI SYSTEMS

Tileable texture synthesis has its own line of work. The Periodic Spatial GAN of Bergmann et al. [9] builds periodic latent fields so that its outputs tile. TileGAN [10] assembles large textures from tiles of a pretrained GAN's latent space, and SeamlessGAN [11] trains a discriminator to judge tiling seams directly. Tiled Diffusion [12] generates diffusion outputs that join at their edges, from single self-tiling textures to many-to-many tile sets. For evaluation, FID [13] compares Inception feature statistics of generated and real images, KID [14] gives an unbiased kernel estimate that suits small samples, CLIP [15] embeddings support zero-shot filters and linear probes, and BLIP [16] generates image captions.

2.4 EXISTING SOLUTIONS / STATE-OF-THE-ART METHODS

Table 2.1: Comparison of existing approaches

Approach	Style control	Layout control	Seamless repeat	Symbol meaning
Text-to-image (SD 1.5) [3]	Weak	None	No	No
LoRA / DreamBooth [4], [5]	Strong	None	No	No
ControlNet [8]	Weak	Strong	No	No
PSGAN / TileGAN / SeamlessGAN [9]-[11]	Texture only	None	Yes	No
Tiled Diffusion [12]	Prompt	Weak	Yes	No
Chhaap (this work)	LoRA	Layout engine	Yes	By construction

2.5 GAP IDENTIFICATION

- No existing system combines a culture-specific style adapter, layout control, seamless repeats and product-specific output.
- Generated images carry no reliable record of which symbols they contain, so any printed explanation may be wrong.
- No method checks whether the symbols requested from a diffusion model actually survived generation.

- Seam metrics are rarely validated, and style metrics are computed against framed paintings rather than repeat designs.
- Cultural ownership is ignored: Warli has held a Geographical Indication since 2014 [17], and GI disputes such as the Gond case [18] show the stakes.

2.6 COMMON FINDINGS FROM LITERATURE

Adapters such as LoRA and ControlNet make large pretrained diffusion models practical for narrow domains without retraining them. Distilled samplers make CPU inference possible. Tileable synthesis is solved for textures but not for structured compositions with meaningful symbols. Automatic image metrics (FID, KID, CLIP scores) are unstable at small sample sizes and must be read alongside visual inspection. Finally, work on cultural heritage repeatedly stresses provenance, labelling and community consent.

CHAPTER 3
REQUIREMENTS SPECIFICATION

3.1 HARDWARE REQUIREMENTS

Table 3.1: Hardware requirements

Component	Minimum	Used in this project
Processor	x86-64 or Apple Silicon, 4 cores	Intel Core Ultra 7 155H
RAM	16 GB	15 GB
GPU (optional)	NVIDIA, 6 GB VRAM	None (CPU only); Colab T4 for training
Storage	10 GB free	About 3.5 GB of model weights plus data

3.2 SOFTWARE REQUIREMENTS

- Operating system: Linux, Windows or macOS.
- Python 3.10 or later with PyTorch, torchvision, diffusers, transformers, accelerate, peft and safetensors.
- FastAPI and uvicorn for the web server; HTML, CSS and JavaScript for the interface.
- Pillow, NumPy, SciPy, scikit-learn, imagehash, torchmetrics and potracer for image processing, metrics and vectorisation.
- SQLite for storing designs and study answers; optional Ollama for a local language-model planner.

3.3 FUNCTIONAL REQUIREMENTS

- FR1: Accept a free-text brief and produce an editable, validated design plan.
- FR2: Lay out lexicon symbols at product size in the chosen template, palette, density and border.
- FR3: Generate seamless tiles for repeating products (wallpaper, saree body).
- FR4: Paint the line work with diffusion (hand-painted mode) or skip it (vector-only mode).
- FR5: Check every placed symbol after diffusion and repair lost hero and support symbols.

- FR6: Export print files at 300 DPI (PNG, SVG) with product mockups and an AI-generated label.
- FR7: Show a meaning card listing present and lost symbols with sources.
- FR8: Show each generative stage ("under the hood") and provide study pages for human evaluation.

3.4 NON-FUNCTIONAL REQUIREMENTS

- Privacy and locality: all generation runs on the user's machine with open weights; no paid API is required.
- Performance: vector-only design in under a second; hand-painted design in seconds on a GPU and a few minutes on a CPU.
- Reproducibility: fixed seeds; every run writes its full configuration and timing to JSON.
- Transparency: every meaning is linked to a source; every export carries an AI-generated label.
- Usability: one-command install and run; browser-based interface.

3.5 GENERATIVE AI MODEL REQUIREMENTS

Table 3.2: Generative models used

Model	Role	Configuration
Stable Diffusion 1.5	Base latent diffusion model	512 px, guidance 1.0
ControlNet (scribble)	Spatial conditioning on the layout	Scale 0.7, +0.15 per repair rerun
LCM-LoRA	Few-step sampling on CPU	6 steps
Warli LoRA (trained)	Warli style	Rank 16, strength 0.8
TAESD	Step previews	Decoder only
CLIP ViT-B/32, BLIP	Data filtering, captions, style probe	Zero-shot and linear probe

3.6 DATASET / KNOWLEDGE-BASE REQUIREMENTS

Style images come from the Kaggle Indian paintings dataset [19] with eight classes (Gond, Kalighat, Kangra, Kerala mural, Madhubani, Mandana, Pichwai and Warli). It states no licence, so it is used for research only. The knowledge base is a symbol lexicon of 25 Warli motifs. Each row has an id, name, local name, meaning, context, source URL and a visual_rule drawing program. Meanings come from six

public sources [1], [2], [20]-[23]. A second lexicon of 14 Mandana motifs supports borders.

CHAPTER 4
SYSTEM DESIGN

4.1 PROPOSED SYSTEM ARCHITECTURE

Chhaap reverses the usual order of generation. Instead of generating a picture and guessing its content, it decides the content first and uses diffusion only for appearance. The system has five layers: a planner that turns the brief into a plan; a layout engine that owns structure, size hierarchy and colour layers; a stylisation stage (Stable Diffusion 1.5, ControlNet, LCM-LoRA and Warli LoRA with circular padding) that paints line work; a survival checker with repair; and a product stage that vectorises, colours, renders mockups and writes the meaning card. A FastAPI server exposes these stages to a browser interface and stores designs in SQLite.

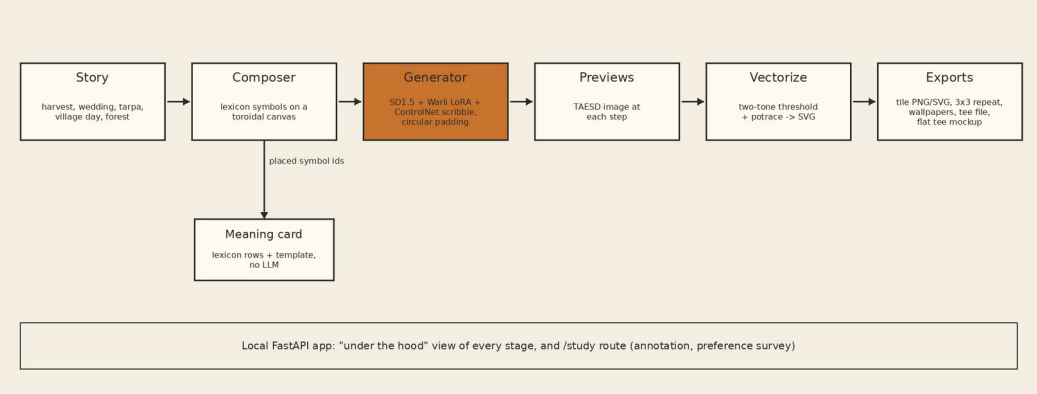


Figure 4.1: Proposed system architecture of Chhaap

4.2 SYSTEM BLOCK DIAGRAM

Figure 4.2 shows the generative blocks for one design, stage by stage: the composed control image produced by the layout engine, the TAESD previews of the six LCM sampling steps, the decoded output and the two-tone reduction used for tracing.



Figure 4.2: Block view of one generation: control image, step previews, decoded tile and two-tone reduction

4.3 SYSTEM WORKFLOW

- Step 1: The user writes a brief or picks an example (for example, "A3 frame for my parents' anniversary, wedding and harvest, maroon and mustard, busy, traditional border").
- Step 2: The planner (keyword rules, or an optional local model) outputs a plan: product, template, density, repeat, palette, border, hero and up to four story beats. Unknown symbol ids are rejected.
- Step 3: The user reviews and edits the plan card.
- Step 4: The layout engine places hero, support, filler and border elements and draws the monochrome ink layer.
- Step 5: In hand-painted mode, diffusion paints the ink layer; otherwise the vector layer is used directly.
- Step 6: The survival check compares each element; lost elements are rerun with a stronger ControlNet, then restored from vector.
- Step 7: The ink is traced to curves, coloured by layer, textured, exported at 300 DPI and placed on mockups with the meaning card.

4.4 MATHEMATICAL MODEL

Let the system be S = {I, P, L, D, C, O}, where I is the brief, P the planner, L the layout engine, D the diffusion stage, C the survival check and O the output. The plan is p = P(I), the layout is a set of elements E = L(p) = {(s_i, l_i, b_i, r_i)} giving symbol, colour layer, box and role, and the control image is the rendering R(E).

Diffusion denoises a latent z_t for t = T, ..., 1 with the noise predictor eps_theta(z_t, t, c_text, c_ctrl), where theta are the frozen Stable Diffusion weights plus the LoRA update W' = W + alpha * B A, with B and A of rank 16. Circular padding makes every convolution operate on a torus, so the decoded tile T satisfies T[:, 0] continuing T[:, W-1] and T[0, :] continuing T[H-1, :].

Survival: for each element i with box b_i, both ink masks are dilated by one pixel and

$$IoU_i = |M_c(b_i) \cap M_o(b_i)| / |M_c(b_i) \cup M_o(b_i)|, \quad survive_i = [IoU_i \geq 0.5]$$

Seam Discontinuity Ratio: on the 2 x 2 mosaic of T, with Sobel gradient magnitude G,

$$SDR = \text{mean}(G \text{ over 4-px seam bands at } x = W,\ y = H) \,/\, \text{mean}(G \text{ over 4-px bands on 8 interior lines})$$

An SDR close to 1 means the join carries no more edge energy than the rest of the tile. Layout evenness is the coefficient of variation of ink coverage over a 4 x 4 grid of the content area: CV = sigma / mu.

CHAPTER 5
PROJECT IMPLEMENTATION

5.1 OVERVIEW OF PROJECT MODULES

The code is organised as a Python package (chhaap) with one module per stage: lexicon and compose (symbols and drawing), layout (templates, lattice repeats, fillers), palette (layers and colour), studio (diffusion pipeline, survival check, repair), planner (brief to plan), vectorize (tracing and export), meaning (meaning cards) and products (product sizes and mockups). A FastAPI application (app.py) serves the interface and the study pages. Scripts prepare data, train the LoRA and run experiments E1 to E10.

5.2 DATA COLLECTION AND PREPROCESSING

The Warli folder of the Kaggle dataset held 97 readable images. Perceptual-hash duplicates (distance 6 or less) were removed (none were found), 17 images with short side under 512 pixels were dropped, and a CLIP zero-shot filter comparing each image against prompts for a Warli painting, a product photograph, a watermarked image and a page of text removed 2 more. A contact-sheet review removed 22 further images the filter missed: framed prints photographed on walls, canvas renders, listings with dimensions printed on them, a watermarked stock image, a craft-kit box and a person posing before a mural.

The remaining 56 images were centre-cropped to 512 x 512 and split 80/20 with a fixed seed into 45 training and 11 held-out images. Captions were generated with BLIP and prefixed with the trigger word warli_style. A manifest records, for every image, its class, source, licence status, size, keep decision, drop reason and split.

5.3 GENERATIVE AI MODEL SELECTION

Stable Diffusion 1.5 was chosen because it is open, has the largest ecosystem of ControlNets and LoRAs, and runs on a CPU at 512 pixels. Larger models (SDXL and later) would not fit the memory and time budget of a 15 GB laptop without a GPU. The scribble ControlNet matches the black-and-white line drawings produced by the layout

engine. LCM-LoRA was selected to reduce sampling to 6 steps with guidance 1.0, which needs one UNet pass per step.

5.4 MODEL CONFIGURATION / FINE-TUNING

The Warli LoRA was trained on a free Colab T4 GPU with the diffusers text-to-image LoRA script: rank 16 on the UNet of Stable Diffusion 1.5, 45 images at 512 pixels, batch size 1, learning rate 1e-4, fp16, 1500 steps (34 epochs) with checkpoints every 500 steps. Training took 10.2 minutes. Experiment E6 compared checkpoints 500, 1000 and 1500 at strengths 0.4, 0.8 and 1.2; the final checkpoint at strength 0.8 was kept because it adds no unplaced figures. The ControlNet scale of 0.7 was chosen by eye on seeds outside the evaluation set: at 0.9 the output turned to black ink on a peach ground, while 0.6 to 0.7 gave white figures on red ochre.



Figure 5.1: LoRA checkpoints (rows: 500, 1000, 1500 steps) against strength (columns: 0.4, 0.8, 1.2)

5.5 PROJECT MODULES

5.5.1 Lexicon and Composer

Each lexicon row has a visual_rule: a short drawing program in symbol-relative units (the symbol spans [-0.5, 0.5]^2) built from discs, polygons, polylines, rectangles, rays, arcs and two composition operators, use (draw another symbol, scaled) and circle_of (draw n copies on a ring). The tarpa dance is therefore a tarpa player inside a ring of 14 human figures. The lexicon is the only definition of each symbol.

5.5.2 Layout Engine

Four templates exist. In the tarpa spiral, dancers hold hands in 2 to 4 concentric rings around the player. In registers, 2 to 4 horizontal bands each carry one story beat. In the medallion, a square frame holds the hero and scenes fill the quadrants. In the all-over buti, small motifs sit on a half-drop grid. Hero, support and filler sizes follow a 8 : 3 : 1 hierarchy, supports never overlap (checked with oriented boxes), and a filler pass adds dots, grass, birds and stars until ink coverage reaches 0.07, 0.12 or 0.17. Repeating products wrap through straight, half-drop or brick lattices, so tiles are periodic by construction.

5.5.3 Layered Colour and Texture

Every element belongs to one of five layers (ground, figure ink, accent, border, filler). Eight palettes assign a colour to each layer, so recolouring is exact. A procedural grain texture is applied to raster outputs only.

5.5.4 Stylisation, Survival Check and Repair

The ink layer is painted by the diffusion pipeline at no more than 768 pixels on its long side, reduced to two tones, and compared element by element with the composition. Lost hero or support elements trigger up to two reruns with the ControlNet scale raised by 0.15, after which the vector symbol is restored inside its box. Repaired ink is traced with Potrace [24] and each path takes the layer of the smallest covering box.

5.5.5 Planner

The planner turns a brief into a typed, validated plan. The default backend is keyword rules; a local Ollama model or a hosted model can be used behind a flag, with one retry and fallback to the rules plan. No backend draws images or writes meanings.

5.5.6 Products, Export and Meaning Card

Products include A4/A3 wall frames, saree sets (body tile, border, pallu), A5 diary wraps, wallpaper, phone wallpapers and tees. Print files are drawn at 300 DPI from vectors and never upscaled. Mockups are drawn from flat shapes with a perspective warp. The meaning card lists each surviving symbol once with count, local name, meaning, context and source URL, and lists lost symbols separately.

5.6 TOOLS AND TECHNOLOGIES USED

Table 5.1: Tools and technologies

Category	Tools
Language	Python 3, JavaScript, HTML, CSS
Deep learning	PyTorch, diffusers, transformers, peft, accelerate
Models	Stable Diffusion 1.5, ControlNet scribble, LCM-LoRA, TAESD, CLIP, BLIP
Image processing	Pillow, NumPy, SciPy, potracer, imagehash
Evaluation	scikit-learn, torchmetrics (KID), SciPy statistics
Backend / storage	FastAPI, uvicorn, SQLite
Training	Google Colab (T4 GPU)
Version control	Git, GitHub

5.7 ALGORITHM DETAILS

5.7.1 Design Generation with Survival Repair

- Input: brief I, seed s, mode (vector or hand-painted).
- 1. p = Plan(I); validate all symbol ids against the lexicon.
- 2. E = Layout(p, s); render ink layer K and colour layers.
- 3. If mode is vector: output = K. Else: Y = Diffuse(K, prompt(E), $s_c = 0.7$).
- 4. For each hero/support element e in E: compute IoU_e between K and Y in box(e).
- 5. While some element is lost and reruns < 2: $s_c = s_c + 0.15$; rerun diffusion; recheck.
- 6. For each element still lost: paste the vector symbol of e into Y inside box(e).

- 7. Trace Y to curves; colour each path by its covering element's layer; export and build meaning card from surviving elements.

5.7.2 Circular Padding

Every 2D convolution in the UNet, ControlNet, VAE and LoRA layers is set to padding_mode = "circular", so the latent is a torus at every resolution and the decoded tile wraps at its edges.

CHAPTER 6
SOFTWARE TESTING

6.1 TEST CASES

Table 6.1: Test cases

ID	Test	Expected result	Status
TC1	SDR self-test on periodic smooth noise	SDR close to 1	Pass
TC2	SDR self-test on shifted tile with ramp	SDR well above 1	Pass
TC3	Planner given unknown symbol id	Plan rejected, rules fallback	Pass
TC4	Layout: supports overlap check	Zero overlapping pairs	Pass
TC5	Half-drop tile wraps	Edges continuous	Pass
TC6	Vector re-rasterisation vs two-tone mask	> 98% pixel agreement	Pass
TC7	Meaning card lists only surviving symbols	Lost symbols listed separately	Pass
TC8	Exports carry AI-generated label	Label present in every file	Pass
TC9	Module self-checks (compose, layout, palette, studio, planner, meaning, vectorize)	All asserts pass	Pass

6.2 GENERATIVE AI OUTPUT TESTING

Generated outputs were tested with fixed seeds (0 to 15, and 1000 upward for version 2) so that every condition is compared on the same content. E1 tests tiling across five conditions with SDR and wrap error. E2 tests style with KID against 11 held-out Warli images and with a CLIP linear probe over eight painting classes. E7 tests symbol survival for every placed symbol over 16 hand-painted pages. E10 runs 20 fixed briefs through the planner and checks the stated constraints automatically. Human annotation (E3) and blind preference (E4) pages are built into the tool.

6.3 PERFORMANCE EVALUATION

Table 6.2: Cost on the test laptop (CPU only), 16 images per condition

Condition	Seconds per image	Denoising (s)	Peak RSS (MB)
a: text only	27.710 ± 0.328	20.921 ± 0.273	7383
b: text + circular	28.365 ± 0.372	21.290 ± 0.347	7645
c: text + LoRA	27.867 ± 0.295	21.065 ± 0.246	7387
d: text + LoRA +	28.469 ± 0.282	21.325 ± 0.236	7646

circular			
e: full pipeline	35.974 ± 0.262	28.778 ± 0.230	7671

A composed tile takes 36.0 seconds on the laptop CPU, of which 28.8 are denoising; the extra time over condition (d) is the ControlNet. LoRA and circular padding each cost under a second. The process peaks at about 7.5 GB, mostly fp32 weights. On an NVIDIA GPU the pipeline switches to fp16 and 25 steps and takes a few seconds per tile.

CHAPTER 7
RESULTS AND DISCUSSION

7.1 PROJECT OUTCOMES

The project delivered a working local design studio that produces complete product designs from a brief: wall frames, saree sets, diary covers, wallpapers, phone wallpapers and tees, in vector-only or hand-painted mode, with 300 DPI exports, mockups and meaning cards. A Warli LoRA was trained and released for research use, a 25-row sourced lexicon was built, and ten experiments were run.



Figure 7.1: Vector-only designs from four briefs: saree set, diary cover, phone wallpapers and a frame with a Mandana border

7.2 RESULT ANALYSIS

Across 16 pages, diffusion kept 0.583 of support symbols (n = 533), 0.750 of heroes (n = 12) and 0.805 of fillers (n = 307) on its first pass. Survival depended on the template: 0.512 in the tarpa spiral, 0.763 in the all-over buti, 0.838 in the medallion and 0.877 in registers. Over the 20 symbols placed at least 3 times, symbols with more drawing primitives survived less often (rho = -0.474, p = 0.0349). In a post-hoc test, rotated symbols survived in 0.468 of 365 cases and upright ones in 0.828 of 180

(Fisher's exact test, odds ratio 0.183, p = 1.51e-16); every rotated symbol is a dancer in a tarpa ring, so rotation is confounded with crowding.

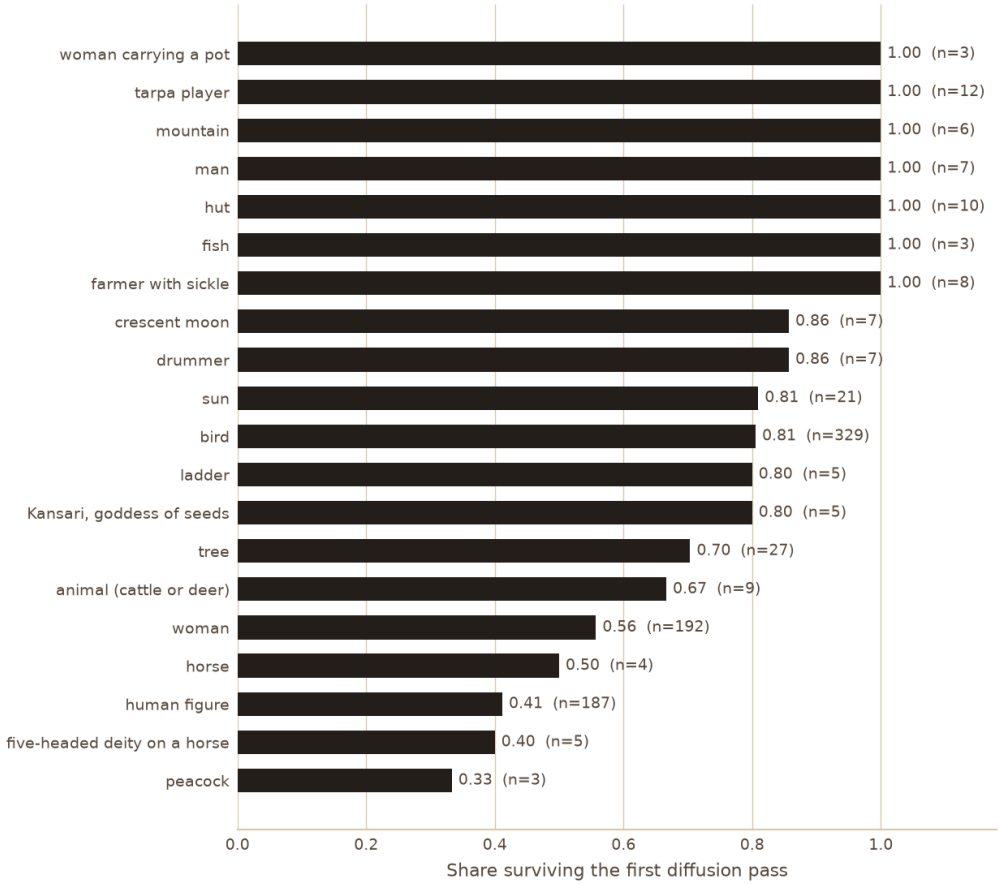


Figure 7.2: Share of each symbol surviving the first diffusion pass over 16 pages

7.3 GENERATED OUTPUT ANALYSIS

Figure 7.3 shows the failure modes: rings of small figures merge into bands and filler dots become stars. Repair recovered every lost hero and support symbol: of 545, 317 passed the first time, 42 were recovered by a rerun with a stronger ControlNet and 186 by restoring the vector symbol. Fillers are not repaired, and 50 of 307 stayed lost.

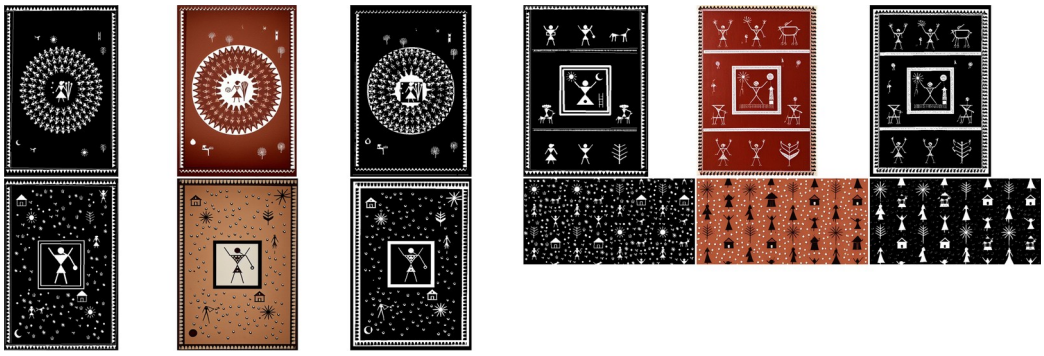


Figure 7.3: Four of the 16 runs: composition, stylised output and repaired ink

For tiling, text-only tiles come out as framed panels, and their 2 x 2 repeat is a grid of frames. With the LoRA and no circular padding the framing is stronger. With circular padding, the outputs are continuous textures, and composed tiles repeat without a visible join (Figure 7.4).

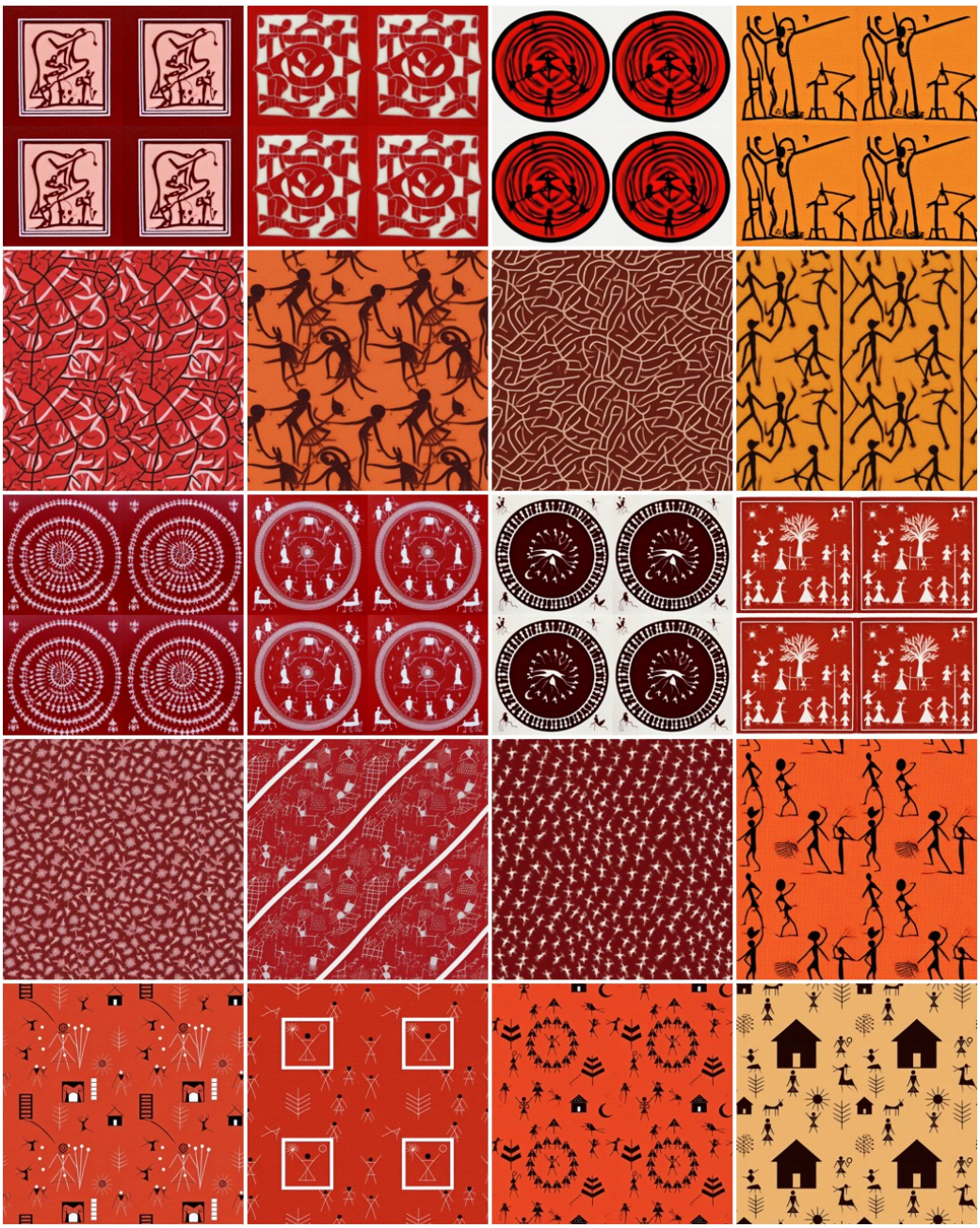


Figure 7.4: 2 x 2 mosaics per condition (rows a to e); seam lines run through the centre of each mosaic

7.4 PERFORMANCE METRICS

Table 7.1: Symbol survival and time per page (16 pages)

Rendering	Hero and support	All symbols	Seconds per page
Vector only	1.000	1.000	0.152 ± 0.075
Stylised	0.587	0.665	67.728 ± 40.398
Stylised + repair	1.000	0.940	141.164 ± 67.535

Vector-only rendering keeps every symbol by construction in a fraction of a second. Diffusion alone loses 0.413 of hero and support symbols. Repair restores all of them and roughly doubles the time, because 10 of 16 pages needed two reruns.

7.5 ACCURACY / RELEVANCE / QUALITY EVALUATION

Table 7.2: Evenness of ink (CV over 4 x 4 grid, lower is more even), 16 seeds

Layout	CV	CV without fillers	Overlapping pairs
Random scatter	1.002 ± 0.126	1.002 ± 0.126	0.125 ± 0.342
Tarpa spiral	0.826 ± 0.040	0.826 ± 0.040	0.000
Registers	0.470 ± 0.107	0.598 ± 0.078	0.000
Medallion	0.504 ± 0.119	1.028 ± 0.052	0.000
All-over buti	0.133 ± 0.029	0.322 ± 0.070	0.000

Every template spread ink more evenly than random scatter (Wilcoxon $p \leq 0.0002$) and none overlapped symbols. Without fillers, registers and the all-over buti remain more even on structure alone, while the medallion's evenness comes from its fillers. For style, the CLIP probe reached 0.806 test accuracy over eight classes; it gives real held-out Warli images a mean Warli probability of 0.301, base text-only tiles 0.169, LoRA tiles 0.324 and composed tiles 0.356. KID ranked LoRA tiles closest (0.024) and composed tiles furthest (0.148), because the reference set consists of dense framed paintings. The rules planner returned a valid plan for all 20 briefs and satisfied 0.944 of 71 stated constraints.

7.6 VALIDATION OF RESULTS

Table 7.3: Tiling, post-hoc |log SDR| (0 is seamless), 16 tiles

Condition	8 lines	64 lines	0.8 < SDR < 1.25
a: text only	0.591 ± 0.492	0.613 ± 0.442	5 of 16
b: text + circular	0.130 ± 0.137	0.107 ± 0.109	13 of 16
c: text + LoRA	1.247 ± 0.937	1.207 ± 0.895	2 of 16
d: text + LoRA + circular	0.147 ± 0.109	0.137 ± 0.090	12 of 16
e: full pipeline	0.279 ± 0.218	0.281 ± 0.159	8 of 16

Results were validated with paired non-parametric tests on fixed seeds. The planned mean SDR did not separate conditions, because a broken seam pushes SDR above 1 while a framed tile pushes it below 1. Using |log SDR| (reported as post hoc), circular padding separates clearly: (b) against (a) gives $W = 10$, $p = 0.0013$, and (d) against (c) gives $W = 2$, $p = 0.0001$. Control images, seamless by construction, still

score 0.586, which shows that SDR is reliable only on dense textures. The human studies (E3, E4) were not run, and the survival correlation was fragile across earlier runs.

7.7 COMPARISON WITH EXISTING APPROACHES

Table 7.4: Comparison with a general text-to-image baseline

Criterion	Text-only SD 1.5	Chhaap
Seamless repeat (|log SDR|)	0.591	0.279 (0.130 on dense tiles)
Warli probability (CLIP probe)	0.169	0.356
Hero/support symbols present	Unknown	1.000 after repair
Meaning card accuracy	Not available	By construction, sourced
Product layouts	None	6 products, 4 templates
Runs locally on CPU	Yes	Yes

7.8 SCREENSHOTS OF THE IMPLEMENTED SYSTEM



Figure 7.5: Chhaap web interface: brief, plan card, press settings and design output



Figure 7.6: The A3 anniversary brief, hand-painted with repair, on a wall-frame mockup

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

Composing symbols before diffusion makes the explanation of an image a by-product of making it, and circular padding makes the tile repeat. Giving the layout engine the structure and colour, and checking each symbol after diffusion, turns a tile generator into a tool that produces complete product designs whose meaning cards list only what is on the page. Diffusion on its own lost 0.413 of the important symbols, above all rotated dancers in crowded rings, and repair recovered all of them at about twice the time. The project shows that a generative model can be used responsibly in a cultural domain when a program owns the content and the model owns only the appearance.

8.2 LIMITATIONS OF THE PROPOSED SYSTEM

- Training images come from a scraped dataset without a licence; only 45 training and 11 evaluation images remain.
- No Warli artist has verified the lexicon, and three meanings rest on a retailer's blog.
- Stable Diffusion 1.5 works at 512 pixels; print exports are sharp only because they are redrawn from vectors, which reduces them to two tones.
- The survival check measures geometry, not meaning: it cannot see a moon redrawn as a sun inside a composite symbol.
- The human studies were not run, and the planner was tested only on our own briefs with the rules backend.
- SDR is unreliable on sparse tiles, and KID is unstable with 11 reference images.

8.3 FUTURE WORK

- Run the annotation and blind preference studies with designers and Warli artists.
- Train an artist-licensed LoRA with a royalty per generated design paid to contributing artists.

- Keep ring dancers upright or larger in the layout engine to reduce symbol loss cheaply.
- Replace SDR's interior reference with a half-width shifted copy of the tile; build a reference set of real Warli repeats.
- Move to a higher-resolution base model on GPU and evaluate the local language-model planner.
- Add trained adapters for other folk styles such as Mandana and Gond, with community consent.

8.4 APPLICATIONS

- Textile and saree print design with seamless body tiles, borders and pallu panels.
- Stationery, home decor, wallpaper and phone wallpaper design.
- Cultural education: meaning cards teach buyers what each motif represents.
- A licensing platform for folk artists to earn royalties from generated designs.
- Rapid prototyping for designers before hand-painted production.

REFERENCES

[1] IIT Bombay D'Source, "Warli Painting - Nagpur, Maharashtra," [Online]. Available: https://www.dsource.in/resource/warli-painting

[2] Wikipedia contributors, "Warli painting," [Online]. Available: https://en.wikipedia.org/wiki/Warli_painting. Accessed: Oct. 22, 2026.

[3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, "High-resolution image synthesis with latent diffusion models," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2022, pp. 10684-10695.

[4] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in Proc. Int. Conf. Learning Representations (ICLR), 2022.

[5] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein and K. Aberman, "DreamBooth: Fine tuning text-to-image diffusion models for subject-driven generation," in Proc. CVPR, 2023, pp. 22500-22510.

[6] S. Luo et al., "LCM-LoRA: A universal Stable-Diffusion acceleration module," arXiv preprint arXiv:2311.05556, 2023.

[7] O. B. Bohan, "TAESD: Tiny AutoEncoder for Stable Diffusion," 2023. [Online]. Available: https://github.com/madebyollin/taesd

[8] L. Zhang, A. Rao and M. Agrawala, "Adding conditional control to text-to-image diffusion models," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2023, pp. 3836-3847.

[9] U. Bergmann, N. Jetchev and R. Vollgraf, "Learning texture manifolds with the Periodic Spatial GAN," in Proc. 34th Int. Conf. Machine Learning (ICML), 2017, pp. 469-477.

[10] A. Fruhstuck, I. Alhashim and P. Wonka, "TileGAN: Synthesis of large-scale non-homogeneous textures," ACM Trans. Graphics, vol. 38, no. 4, 2019.

[11] C. Rodriguez-Pardo and E. Garces, "SeamlessGAN: Self-supervised synthesis of tileable texture maps," IEEE Trans. Visualization and Computer Graphics, 2023.

[12] O. Madar and O. Fried, "Tiled Diffusion," in Proc. CVPR, 2025.

[13] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[14] M. Binkowski, D. J. Sutherland, M. Arbel and A. Gretton, "Demystifying MMD GANs," in Proc. ICLR, 2018.

[15] A. Radford et al., "Learning transferable visual models from natural language supervision," in Proc. 38th ICML, 2021, pp. 8748-8763.

[16] J. Li, D. Li, C. Xiong and S. Hoi, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in Proc. 39th ICML, 2022, pp. 12888-12900.

[17] The Better India, "How Adivasi women reclaim Warli art for international recognition with murals, GI tags," Sep. 2022. [Online]. Available: https://thebetterindia.com/296974/

[18] People's Archive of Rural India, "Gond art: stories of the forest and the land," [Online]. Available: https://ruralindiaonline.org

[19] ajg117, "Indian Paintings Dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/ajg117/indian-paintings-dataset

[20] M. Swaminathan, "From Wall to Walls: the Warli paintings," IGNCA. [Online]. Available: https://ignca.gov.in/?p=13011

[21] A. Mishra, "The enduring legacy of the Warli tribe: guardians of Maharashtra's living folk tradition," Organiser, Nov. 2025.

[22] Sarmaya, "Kansari (Goddess of Seeds), by Tushar and Mayur Vayeda," [Online]. Available: https://sarmaya.in/objects/indigenous-tribal-art/kansari-goddess-of-seeds/

[23] MeMeraki, "Warli paintings, different types and styles," [Online]. Available: https://www.memeraki.com

[24] P. Selinger, "Potrace: a polygon-based tracing algorithm," 2003. [Online]. Available: https://potrace.sourceforge.net/potrace.pdf

APPENDIX A: DETAILS OF PAPER PUBLICATION

Draft paper: “Chhaap: Layout-first diffusion with symbol survival checks for Warli-inspired product design”, K. Shetshringi, A. Kanchankar, S. Khillare, P. Kolettiwar and P. Nikumbh (Dhade), prepared in IEEE conference format (paper/Chhaap_Paper.pdf in the project repository). Conference submission details, reviewer comments and certificates to be attached.

Source code: https://github.com/AadiKanchankar/chhaap

APPENDIX B: PLAGIARISM REPORT

Plagiarism check report of this project report, verified by the guide, to be attached here.